CLAUDE CERTIFIED ARCHITECT — PROFESSIONAL

Objective 5: Apply Task Decomposition Techniques

Exam objective: Apply task decomposition techniques by breaking complex tasks into independent subtasks, identifying dependencies between them, defining clear contracts between components, ensuring each step is independently testable, and deciding which subtasks should run in parallel versus sequentially.

Q1 **An architect is decomposing a mortgage pre-approval workflow into subtasks: (1) extract applicant income data, (2) extract credit report data, (3) compute debt-to-income ratio, (4) generate pre-approval letter. Which pair of steps has a hard dependency that must be respected?**

A Steps 1 and 2 depend on each other and cannot run in parallel

B **Step 3 depends on the outputs of steps 1 and 2, and step 4 depends on step 3**

C Step 4 can run before step 3 to save time

D All four steps are fully independent and can run in any order

CORRECT ANSWER **B**

WHY Computing a debt-to-income ratio requires both income and credit data to already be extracted, and the final letter requires the computed ratio, so this chain reflects genuine data dependencies that must be respected in execution order. A incorrectly asserts a dependency between two tasks that are actually independent of each other (extraction of income versus credit data), and C and D ignore real dependencies.

Q2 **A retail company is decomposing a product-catalog enrichment task across 50,000 independent SKUs, each needing a description generated from its own spec sheet. What is the BEST decomposition strategy?**

A Process all 50,000 SKUs within a single Claude call

B **Decompose into independent per-SKU (or batched) subtasks that can run in parallel, since each SKU's enrichment does not depend on any other SKU's result**

C Process SKUs sequentially one at a time to avoid any possible error

D Decompose by letter of the alphabet of the product name regardless of actual content

CORRECT ANSWER B

WHY Since each SKU's enrichment is informationally independent of every other SKU, decomposing into parallelizable per-SKU or batched subtasks is both correct and efficient, dramatically reducing total latency. A would likely exceed context limits and conflate unrelated items, C needlessly sacrifices available parallelism, and D decomposes along an arbitrary axis unrelated to the actual independence structure of the work.

Q3 **A healthcare analytics team is decomposing a patient risk-scoring pipeline. Which decomposition reflects a well-defined "contract" between two components?**

A "The extraction step outputs some text, and the scoring step figures out what to do with it"

B **"The extraction step outputs a structured JSON object with named fields (age, diagnosis_codes, lab_values) and explicit types, which the scoring step consumes according to that exact schema"**

C "The extraction step and scoring step share a single combined prompt with no defined boundary"

D "The scoring step guesses the format based on whatever the extraction step happened to produce that day"

CORRECT ANSWER **B**

WHY A clear contract specifies an explicit, versioned interface — in this case a structured schema with named, typed fields — so each component can be built, tested and changed independently as long as the contract holds. A, C and D all describe ambiguous or undefined interfaces that make independent testing and evolution unreliable.

Q4 A logistics company wants each subtask in its route-optimization decomposition to be independently testable. Which design BEST achieves this?

A **Each subtask is a self-contained function with clearly defined inputs and outputs, allowing it to be tested in isolation with mock data**

B All subtasks share global mutable state so none can be tested without running the entire pipeline

C Subtasks are defined only informally in a team meeting with no written interface

D Testing is deferred until the entire system is complete and deployed to production

CORRECT ANSWER A

WHY Independent testability requires subtasks to have clearly scoped inputs and outputs so each can be validated in isolation using mock or fixture data before full integration. B creates hidden coupling that prevents isolated testing, C lacks the documented interface needed for reliable testing, and D defers critical quality feedback far too late.

Q5 An architect decomposing a legal contract analysis task identifies two subtasks: "extract all dollar amounts mentioned" and "extract all named parties." Why is it reasonable to run these in parallel?

A Because legal documents are always short

B **Because the two extraction subtasks do not depend on each other's output and both can be derived independently from the same source document**

C Because dollar amounts are more important than party names

D Because parallel execution is always faster regardless of dependencies

CORRECT ANSWER B

WHY The defining criterion for parallelizing two subtasks is the absence of a data dependency between them; both extractions read from the same source but produce unrelated outputs, making concurrent execution safe and beneficial. A and C are irrelevant to the dependency analysis, and D is a false generalization since parallelizing truly dependent tasks would produce incorrect results.

Q6 **A manufacturing company's defect-reporting workflow has these candidate subtasks: (1) transcribe inspector's voice note, (2) classify defect severity from the transcription, (3) look up the part's warranty status from a database, (4) generate a combined report. Which two subtasks can safely run in parallel?**

A Subtasks 1 and 2, since both involve the voice note

B Subtasks 2 and 3, since classifying severity from the transcription and looking up warranty status are independent of each other once the transcription exists

C Subtasks 3 and 4, since the report needs the warranty lookup first

D No subtasks can run in parallel in this workflow

CORRECT ANSWER B

WHY Once the transcription (step 1) is available, severity classification and the warranty database lookup draw from independent inputs and do not depend on each other, making them safe to parallelize before the final report assembly. A incorrectly treats a sequential dependency (2 needs 1's output) as parallelizable, C has a genuine dependency (4 needs 3's result) that forbids parallel execution, and D misses real parallelization opportunity.

Q7 **Which indicator BEST signals that two subtasks have a dependency that prevents parallel execution?**

A Both subtasks use Claude as the underlying model

B One subtask requires, as an input, the output produced by the other subtask

C Both subtasks take longer than five seconds to complete

D Both subtasks were written by the same engineer

CORRECT ANSWER B

WHY A genuine dependency exists precisely when one subtask's required input is the other's output; this data-flow relationship is the correct basis for sequencing decisions, not surface factors like shared tooling, duration, or authorship, which are irrelevant to dependency analysis (A, C, D).

Q8 **An insurance company is decomposing a claims intake process and debating whether "validate claim form completeness" should be a separate subtask from "extract claim details." What is the BEST justification for separating them?**

A Separation makes the diagram look more sophisticated

B **Completeness validation is a simple deterministic check (are required fields present) distinct from the nuanced language-understanding task of detail extraction, so separating them allows each to be built, tested and potentially implemented with different tools independently**

C They must always be combined into a single step for efficiency

D There is no meaningful difference between the two tasks

CORRECT ANSWER **B**

WHY These are genuinely different kinds of work — deterministic presence checking versus language-based extraction — and separating them allows each to be implemented with the most appropriate approach and tested independently, which is the core value of good decomposition. A is a cosmetic non-reason, and C and D understate the real distinction between the two subtasks.

Q9 **A telecom company decomposes a network-outage root-cause analysis into: (1) gather logs from three unrelated systems, (2) correlate timestamps across all three log sets, (3) draft a customer-facing outage summary. Which subtasks can run in parallel, and which must wait?**

A. All three subtasks can run in parallel with no restriction

B. **Subtask 1's three log-gathering calls (one per system) can run in parallel with each other, but subtask 2 must wait for all of subtask 1 to complete, and subtask 3 must wait for subtask 2**

C. Subtask 3 can run before subtask 2 to save time

D. Subtasks 1 and 2 must run strictly one log source at a time with no parallelism anywhere

CORRECT ANSWER B

WHY The three log-gathering calls within subtask 1 are independent of each other and can be parallelized, but correlation (subtask 2) requires all logs to exist first, and the summary (subtask 3) requires the correlated result, reflecting the correct mixed parallel/sequential structure. A ignores real dependencies, C violates the data dependency, and D fails to exploit legitimate parallelism within subtask 1.

Q10 **A government agency wants each component in its decomposed benefits-eligibility pipeline to be independently testable by a QA team. What should the architect require as part of each component's definition?**

A. A shared mutable database table with no access boundaries

B. A documented input schema, output schema, and expected behavior for edge cases, independent of the internal implementation details of other components

C. A requirement that all components be tested only together, never individually

D. No documentation, since Claude's behavior is self-explanatory

CORRECT ANSWER B

WHY Independent testability requires an explicit contract — defined inputs, outputs and edge-case behavior — that a QA team can validate without needing to understand or run the rest of the pipeline. A creates hidden coupling, C defeats the purpose of decomposition, and D leaves no artifact for QA to test against.

Q11 **A SaaS company is decomposing an onboarding-document-processing task. They initially define one giant subtask: "process the new customer's uploaded documents." What is the PRIMARY problem with this decomposition?**

A It is too granular and should be combined with other tasks

B **It bundles multiple distinct responsibilities (format validation, data extraction, duplicate detection, storage) into one opaque unit, making it hard to test, parallelize or reason about failures independently**

C It uses too many tokens because it is a single subtask

D It cannot use Claude at all

CORRECT ANSWER B

WHY A single undifferentiated "do everything" subtask hides multiple distinct concerns that should be decomposed further so each can be tested, assigned the right tool, and parallelized where appropriate; this is the opposite problem of over-decomposition. A mischaracterizes the issue (it is under-decomposed, not over-decomposed), and C and D are not accurate critiques of the described design.

Q12 An architect decomposing a manufacturing predictive-maintenance task defines a contract where the anomaly-detection subtask must output: `{sensor_id: string, anomaly_type: enum, confidence: float, timestamp: ISO8601}`. What is the PRIMARY benefit of this explicit contract?

A. It guarantees the anomaly detection will always be 100% accurate

B. It allows the downstream recommendation-generation subtask to be developed, tested and changed independently, as long as it continues to consume this agreed schema

C. It eliminates the need for any error handling downstream

D. It allows the two subtasks to be merged into one in the future with no consequence

CORRECT ANSWER B

WHY A well-defined schema contract decouples the two subtasks so each can evolve independently as long as the interface is honored, which is the central purpose of defining contracts in decomposition. A conflates a data contract with model accuracy, and C and D are unsupported claims not implied by having a schema.

Q13 **A legal firm's contract-review decomposition has a step "flag non-standard clauses" that depends on a prior step "classify contract type," which itself depends on "extract document text." An architect proposes running all three in parallel to save time. What is wrong with this proposal?**

A Nothing is wrong; parallelizing dependent steps is always safe

B **Each step requires the prior step's output as an input, so running them in parallel would cause later steps to execute on missing or stale data, producing incorrect or failed results**

C Running them in parallel would save no time at all, so it does not matter

D Legal documents cannot be processed in multiple steps

CORRECT ANSWER **B**

WHY This is a sequential dependency chain where each step genuinely needs the prior step's completed output; parallelizing it would violate correctness since downstream steps would start before their required inputs exist. A is a false general claim, C understates the correctness risk (which matters far more than time savings here), and D is an irrelevant and incorrect claim.

Q14 Which of the following subtask pairs in an airline operations platform is the BEST candidate for parallel execution?

A **"Check weather delays for flight 204" and "check crew duty-time limits for flight 204," when the final rebooking decision needs both but neither depends on the other**

B "Rebook the passenger" before "confirm the passenger's new seat is available"

C "Generate the final itinerary" before "gather the passenger's rebooking preferences"

D "Charge the change fee" before "confirm the new flight has been booked"

CORRECT ANSWER A

WHY Weather delay and crew duty-time checks draw from independent data sources and do not depend on each other's results, making them safe to run concurrently before being combined for the final decision. B, C and D each describe a step being proposed before a prerequisite it actually depends on, which is an ordering error rather than a valid parallelization opportunity.

Q15 An architect is reviewing a decomposition for a retail returns-processing pipeline and notices that "validate return eligibility" and "calculate refund amount" are combined into a single subtask. A stakeholder asks why these should be split. What is the BEST answer?

A. They should not be split; combining them is more efficient

B. Splitting them creates two independently testable units with distinct contracts (an eligibility boolean plus reason code, and a computed refund amount), allowing each rule set to change independently without risking regressions in the other

C. Splitting them would require two separate teams to be hired

D. Splitting them makes the pipeline slower with no benefit

CORRECT ANSWER B

WHY These represent two distinct business rule sets (eligibility policy versus refund calculation) that commonly change independently; splitting them into separately testable units with clear output contracts reduces the blast radius of future changes and improves testability. A, C and D all undervalue the maintainability and correctness benefits of proper separation.

Q16 **Six months after launch, a manufacturing company's defect-reporting pipeline (transcribe → classify severity → look up warranty → generate report) is handling twice the original volume, and the classification subtask's accuracy has dipped slightly on an edge-case defect type. Separately, a new regulatory requirement now mandates that every report include a traceable chain-of-custody record naming every system and person who touched the defect data, a concept the current decomposition has no component or contract to express. Which situation requires redesigning the decomposition, and which is iteration?**

A Both situations require a full redesign, since any change to a live pipeline is a redesign

B The volume increase and accuracy dip are both absorbable as iteration within the current components; the chain-of-custody requirement forces redesign, because no existing subtask or contract has a place to carry that information

C Both situations are iteration, since the pipeline's four subtasks remain conceptually the same

D The chain-of-custody requirement is iteration, since it can simply be appended as a note in the final report's text

CORRECT ANSWER **B**

WHY A signal forces redesign only when it names a requirement the current decomposition genuinely cannot express; volume growth and an accuracy dip on an edge case are both absorbable inside the existing subtask boundaries and are therefore iteration, while a chain-of-custody requirement introduces a new concept with no owning component or contract, which requires adding or restructuring parts of the decomposition. A overclassifies routine changes as redesign, C misses the genuine new requirement, and D treats a structural, auditable requirement as a cosmetic text addition rather than something that needs its own contract. --